

Distributed Systems

Lecture4: Architectural Styles

Mohammad Reza Shakournia

shakournia@ut.ac.ir

School of Electrical and Computer Engineering,
University of Tehran,
Tehran, Iran

Architectural Styles

- A style is formulated in terms of:
  - (replaceable) components with well-defined interfaces
  - the way that components are connected to each other
  - the data exchanged between components
  - how these components and connectors are jointly configured into a system.

Definition 2.1. Connector

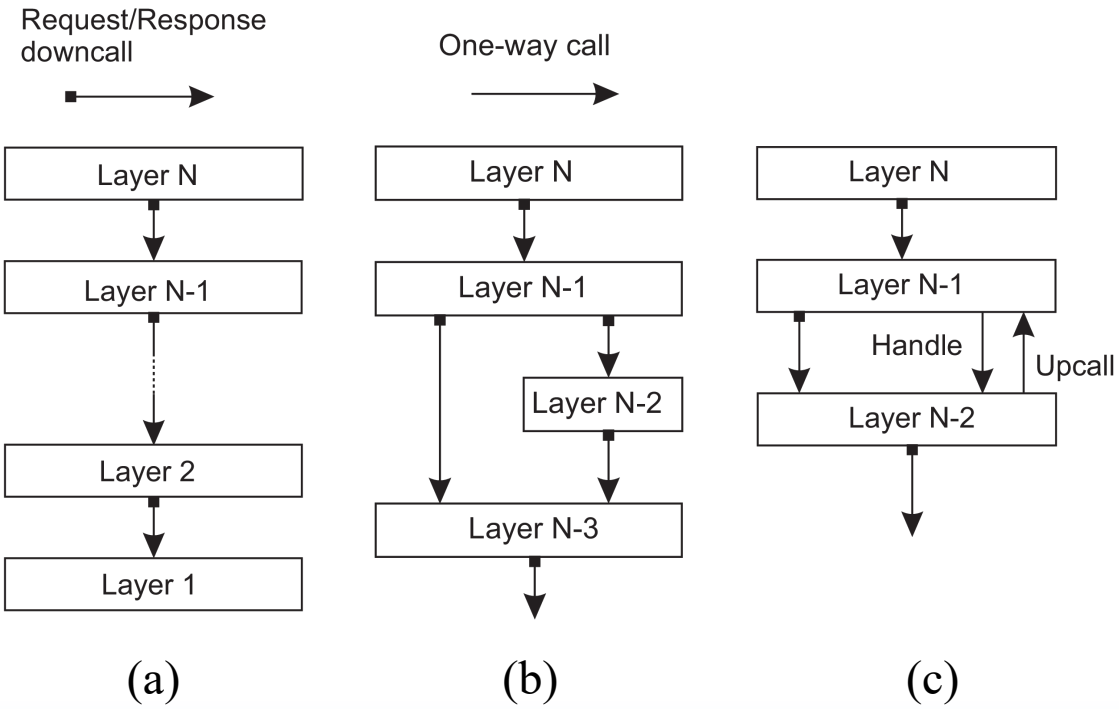
A mechanism that mediates communication, coordination, or cooperation among components.

Example: facilities for (remote) procedure call, messaging, or streaming.

S1. Layered Architectures

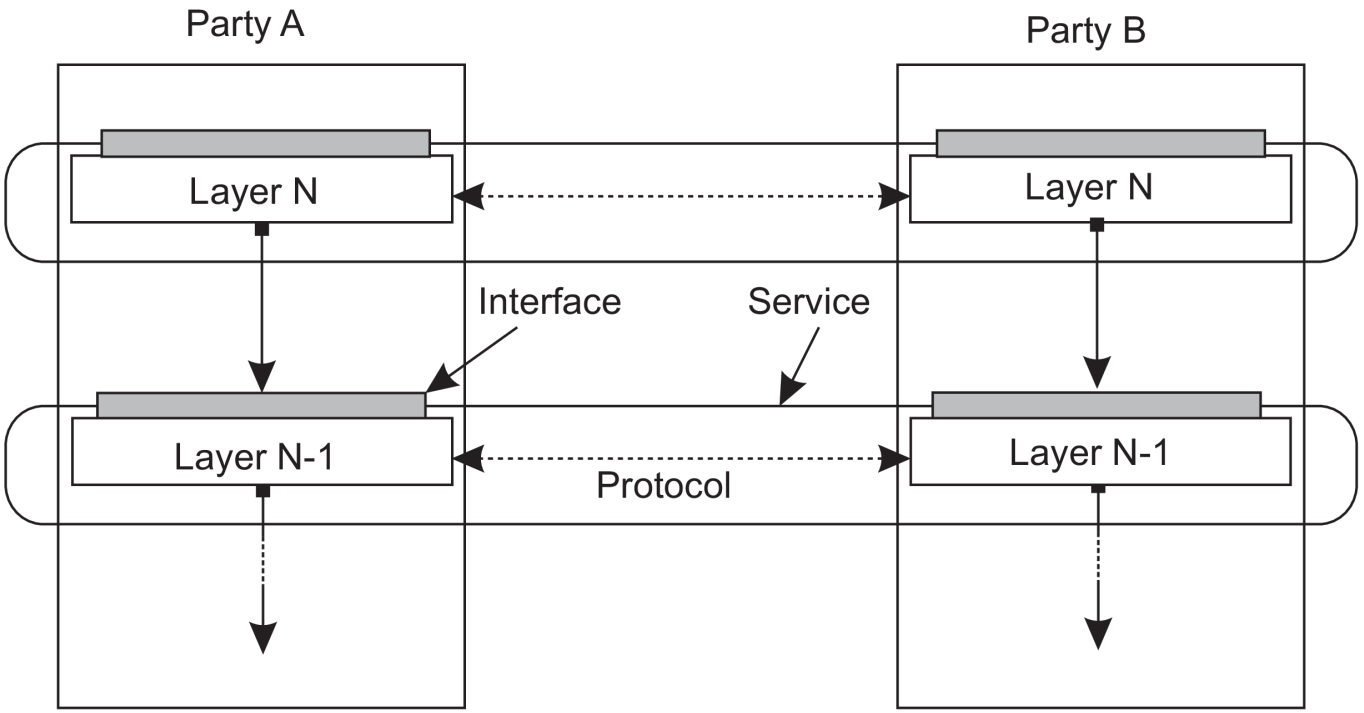
- Core idea: organize components into layers.

  (a) Pure layered organization.
  (b) Mixed layered organization.
  (c) Layered organization with upcalls



Layered Communication Protocols

- A well-known and ubiquitously applied layered architecture is that of communication-protocol stacks.



Example: Two Communicating Parties

```
1  from socket  import *
2
3  s = socket(AF_INET, SOCK_STREAM)
4  (conn, addr) = s.accept()  # returns new socket and addr. client
5  while True:                # forever
6    data = conn.recv(1024)   # receive data from client
7    if not data: break       # stop if client stopped
8    msg = data.decode()+"*"  # process the incoming data into a response
9    conn.send(msg.encode())  # return the response
0  conn.close()               # close the connection
```

(a) A simple server

```
from socket  import *

s = socket(AF_INET, SOCK_STREAM)
s.connect((HOST, PORT)) # connect to server (block until accepted)
msg = "Hello World"     # compose a message
s.send(msg.encode())    # send the message
data = s.recv(1024)     # receive the response
print(data.decode())    # print the result
s.close()               # close the connection
```

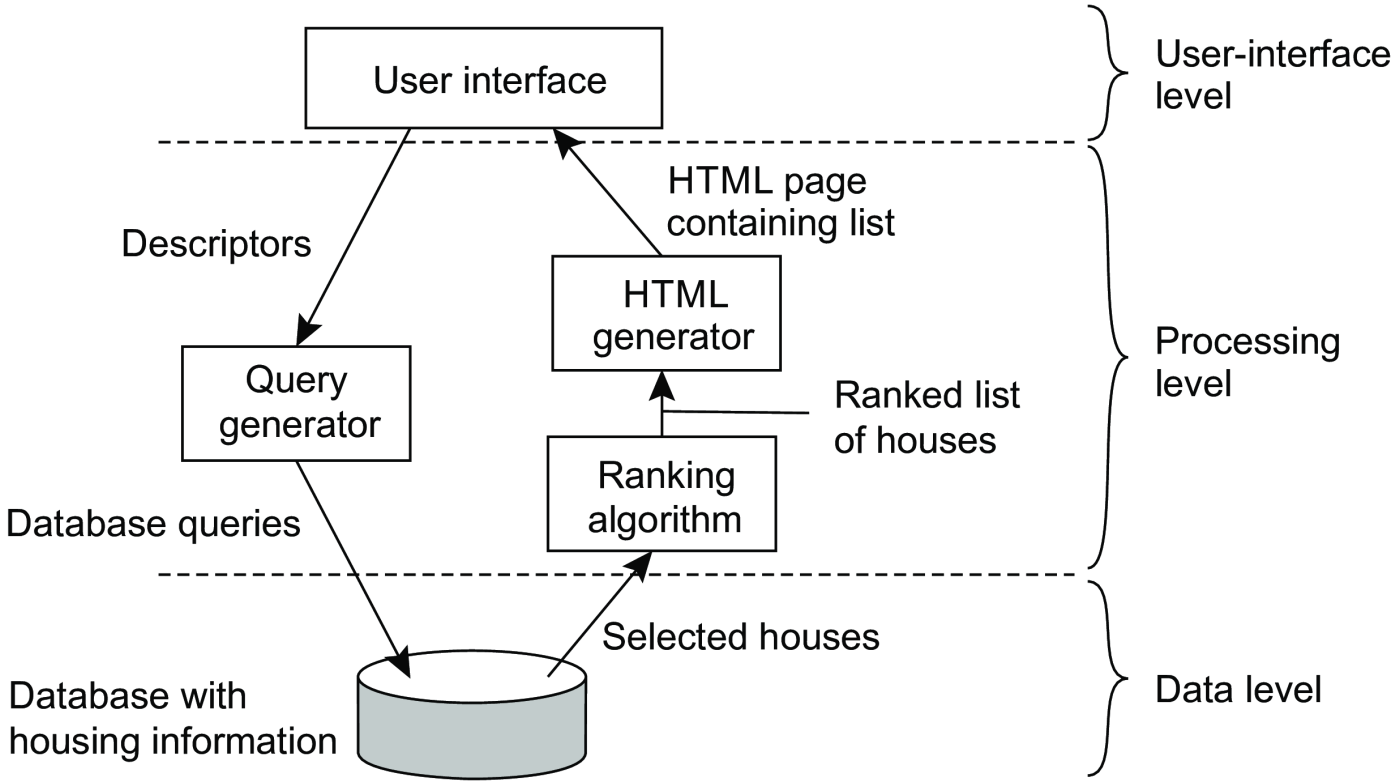
(b) A client

Application Layering

- Many distributed applications support users/applications accessing databases, so a common logical layering uses three levels:
  - **Application-interface level:** contains units for interfacing to users or external applications.
  - **Processing level:** contains the functions of an application, i.e., without specific data.
  - **Data level:** contains the data that a client wants to manipulate through the application components.
- With this layering, an application is often built from three main parts:
  - **Interface part**: handles interaction with users or external applications
  - **Data part**: interacts with a database or file system
  - **Middle/core part**: implements the main logic → processing level
- The processing level varies widely across applications (less “common” structure than UI or databases), so examples help clarify it.

Application Layering

- Example : Internet search engine (e.g., house listings)

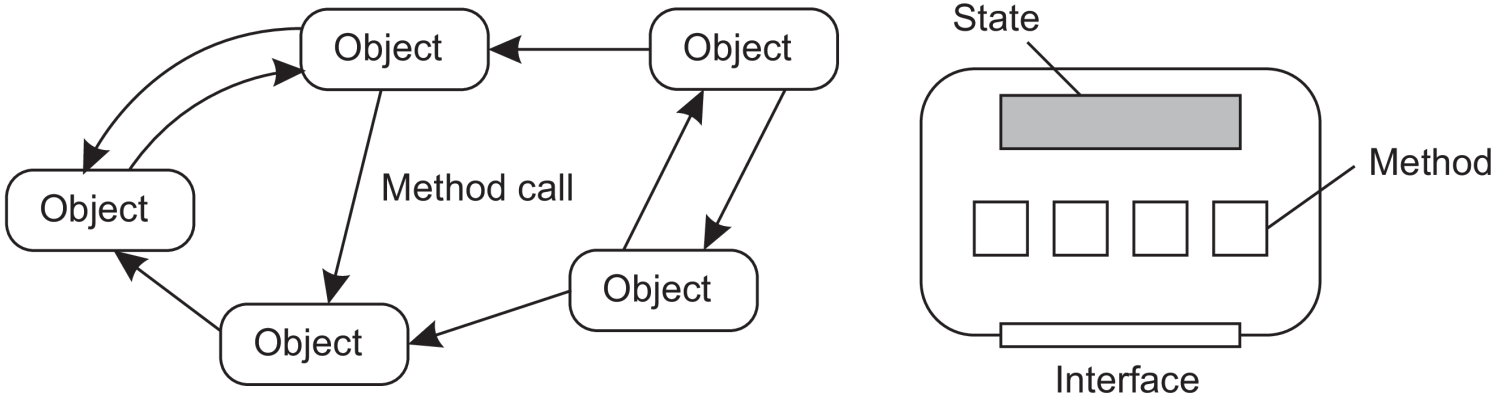


S2. Service-Oriented Architectures

- One of the major drawbacks of the Architecture Layer is the strong dependency between different layers.
  - Good examples: communication protocol stacks.
  - Bad examples: applications developed by different components without concern for the stability of interfaces.
- In service-oriented architecture, each entity encapsulates a service.
- These services could be called as a **service**, **object**, or **microservice**, but they have in common that all of them are executed as a separate process or thread.

Object-based Architectural Style

- In this style, each object corresponds to what we have defined as a component.
- Objects are connected through procedure-call/remote procedure-call
- In a distributed system, the calling object need not be executed on the same machine as the called object.
- In fact, where exactly the called object is located can be transparent to the caller.



Microservice Architectural Style

- Many small, independent programs can be composed to form a larger program.
- Therefore, a service can be built from multiple **microservices**.
- Essential is that each microservice runs as a separate (network) process.
- The implementation of a microservice could be in the form of a **remote object**.
- A distributed application can be constructed from many services, and not all of them may belong to the same administrative organization.
  - For example, a business may use cloud storage services provided by a cloud provider.
- Microservices are relatively small, whereas a general service is typically larger. It is not uncommon to implement a service as a collection of microservices.

Microservice Architectural Style

- Microservices are relatively small, whereas a general service is typically larger.
- It is not uncommon to implement a service as a collection of microservices.
- The problem of developing a distributed system is partly one of service composition and making sure that those services operate in harmony.
  - The enterprise application integration!
  - Each service must offer a well-defined (programming) interface!
- Unfortunately, in practice, each service has its own interface, making composition far from trivial.

Resource-based Architectures

- One of the problems with service composition is that connecting various components can easily turn into an integration nightmare.
- As an alternative, a distributed system could be a huge collection of resources that are individually managed by components.
- Resources may be added or removed by (remote) applications, and likewise can be retrieved or modified.
- This approach is adopted widely for web and known as Representational State Transfer (REST).

Resource-based Architectures

- There are four key characteristics of what are known as **RESTful** architectures:

  ① Resources are identified through a single naming scheme.

  ② All services offer the same interface, consisting of at most four operations.

  ③ Messages sent to or from a service are fully self-described.

  ④ After executing an operation at a service, that component forgets everything about the caller.

Operation	Description
PUT	Modify a resource by transferring a new state
POST	Create a new resource
GET	Retrieve the state of a resource in some representation
DELETE	Delete a resource

- Example: Amazon S3.

Example: Amazon's Simple Storage Service

- Objects (i.e., files) are placed into buckets (i.e., directories). Buckets cannot be placed into buckets. Operations on *ObjectName* in bucket *BucketName* require the following identifier:
- `http://BucketName.s3.amazonaws.com/ObjectName`
- All operations are carried out by sending HTTP requests:
  - Create a bucket/object: PUT, along with the URI
  - Listing objects: GET on a bucket name
  - Reading an object: GET on a full URI

On Interfaces

- Many people like RESTful approaches because the interface to a service is so simple.
- The catch is that much needs to be done in the parameter space.
- Amazon S3 SOAP interface:

Bucket operations	Object operations
ListAllMyBuckets	PutObjectInline
CreateBucket	PutObject
DeleteBucket	CopyObject
ListBucket	GetObject
GetBucketAccessControlPolicy	GetObjectExtended
SetBucketAccessControlPolicy	DeleteObject
GetBucketLoggingStatus	GetObjectAccessControlPolicy
SetBucketLoggingStatus	SetObjectAccessControlPolicy

On Interfaces

- Simplifications:
  - Assume an interface bucket offering an operation create, requiring an input string such as *mybucket*, for creating a bucket "*mybucket*."
- **SOAP:**

```
import bucket
bucket.create("mybucket")
```

- **RESTful:**

```
PUT "https://mybucket.s3.amazonsws.com/"
```

S3. Publish-subscribe Architectures

- As systems continue to grow and processes can more easily join/leave, it becomes important to reduce dependencies between processes as much as possible.
- To achieve this, it is necessary to separate processing from coordination.
- The idea is to view a system as a collection of autonomously operating processes.
- However, coordination is required to ensure that the system operates as a unified whole.
- Coordination encompasses communication and cooperation between processes.

Coordination Models

- When selecting a coordination model, we need to take these two dimensions into account.
- **Temporal Coupling**: Do both communicating processes need to be up and running at the same time?
- **Referential Coupling:** Does a process need to know the name or identifier of another process before communication?

	Temporally coupled	**Temporally decoupled**
Referentially coupled	Direct	Mailbox
Referentially decoupled	Event-based	Shared data space

Coordination Models

① **Direct Coordination:** Both processes must be active and know each other.

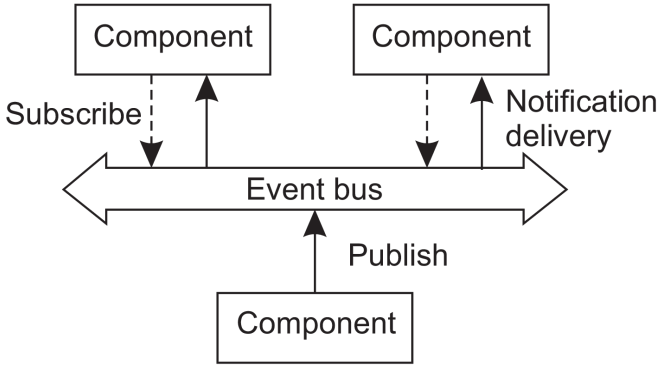
② **Mailbox Coordination:** No need to be active, but must know the address of the mailbox holding the message.

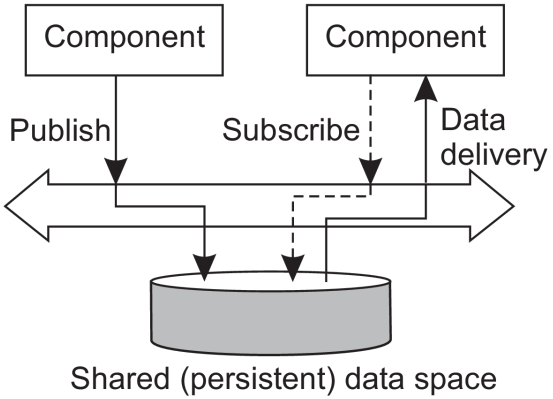
③ **Event-Based Coordination:** Processes do not know each other explicitly, but it is important that the subscriber be up and running when the message/notification is published.

④ **Shared Data Space:** Processes communicate by putting any type of tuple into the shared data space, and the other process provides a search pattern that is matched against the tuples.

Event-based vs Shared Data Space

- Shared data spaces are often combined with event-based coordination:
  - A process subscribes to certain tuples by providing a search pattern;
  - When a process inserts a tuple into the data space, matching subscribers are notified.
- In both cases, we are dealing with a publish-subscribe architecture, and the processes do not know each other.





Example: Linda Tuple Space

- Three simple operations:
  - in(t): remove a tuple matching template t
  - rd(t): obtain copy of a tuple matching template t
  - out(t): add tuple t to the tuple space.
- Calling *out(t)* twice in a row, leads to storing two copies of tuple t⇒ a tuple space is modeled as a multiset.
- Both *in* and *rd* are blocking operations: the caller will be blocked until a matching tuple is found, or has become available.

Example: Linda Tuple Space

```
blog = linda.universe._rd(("MicroBlog",linda.TupleSpace))[1]

blog._out(("bob","distsys","I am studying chap 2"))
blog._out(("bob","distsys","The linda example's pretty simple"))
blog._out(("bob","gtcn","Cool book!"))
```

(a) Bob's code for creating a microblog and posting three messages.

```
blog = linda.universe._rd(("MicroBlog",linda.TupleSpace))[1]

blog._out(("alice","gtcn","This graph theory stuff is not easy"))
blog._out(("alice","distsys","I like systems more than graphs"))
```

(b) Alice's code for creating a microblog and posting two messages.

```
blog = linda.universe._rd(("MicroBlog",linda.TupleSpace))[1]

t1 = blog._rd(("bob","distsys",str))
t2 = blog._rd(("alice","gtcn",str))
t3 = blog._rd(("bob","gtcn",str))
```

(c) Chuck reading a message from Bob's and Alice's microblog.

Publisher-subscriber Principles

Definition 2.2. Event

An occurrence (or condition) of interest in the system.

Example: “Door unlocked”, “Temperature = 40°C”.

Definition 2.3. Notification

A message published to announce that an event has occurred.

Example: {event: "door_unlocked", room: "ZI.1060", time: "10:15"}.

Definition 2.4. Subscription

A request to receive notifications for events that match a description.

Example: “Notify me when room=ZI.1060 AND door=unlocked”.

Definition 2.5. Description

The specification that defines an event or a subscription

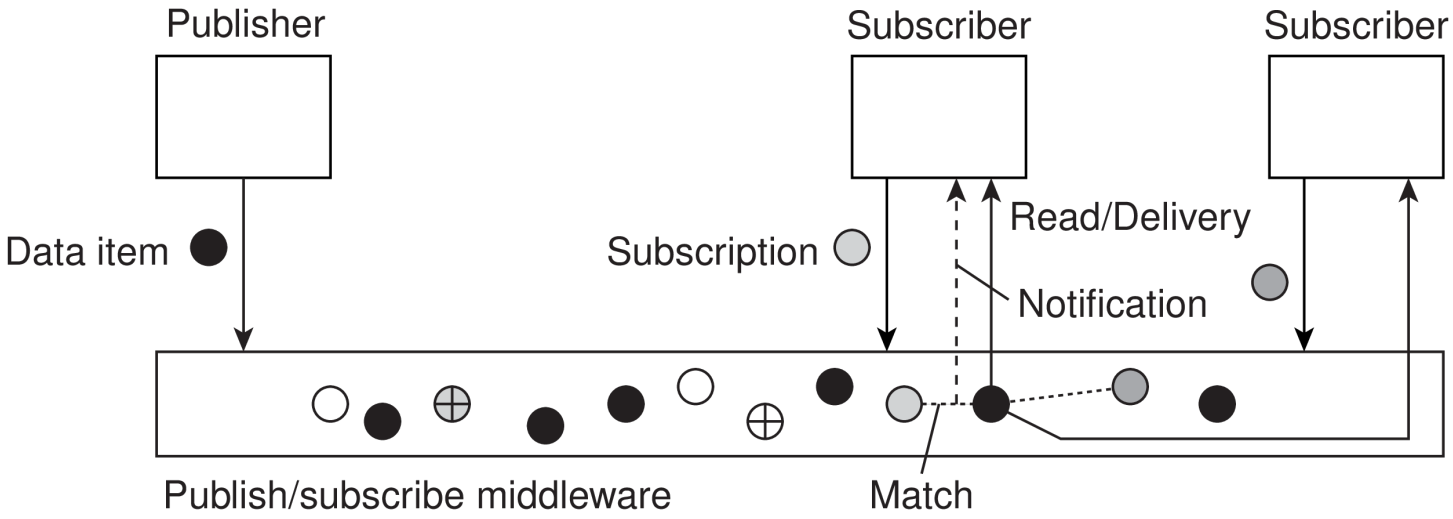
Example: (temperature > 30), (type="door_unlocked", room="ZI.1060")

Publisher-subscriber Principles

- An important aspect of publish-subscribe systems is that communication takes place by describing the events that a subscriber is interested in.
- Naming plays a crucial role, but the important issue is that often, data items are not explicitly identified by senders and receivers.
- Assume events are described by **a series of attributes**.
- A notification describing an event is **published** when it is made available for other processes to read.
- A subscription needs to be passed to the middleware, containing a description of the event that the subscriber is interested in.
- Subscription descriptions:
  - Topic-based publish-subscribe
  - Content-based publish-subscribe

Publisher-subscriber Principles

- Issue: how to match events?
  - Assume events are described by (attribute,value) pairs
  - **Topic-based Subscription:** specify a “attribute = value” series
  - **Content-based subscription:** specify a “attribute ∈ range” series



Wrap-Up + What's Next

In the next lecture, we will discuss middleware and distributed systems context such as:

- Middleware organization,
- Modifiable middleware.